```
    @Test
    public void should_earn_new_status_based_on_point_thresholds() {
        FrequentFlyer member
                    = FrequentFlyer.withFrequentFlyerNumber("12345678")
                                   .named("Joe", "Jones")
                                   .withStatusPoints(initialPoints)
                                   .withStatus(initialStatus);

        member.earns(earnedPoints).statusPoints();

        assertThat(member.getStatus()).isEqualTo(finalStatus);

    }
}
```

The test is run once for each row of data.

More modern unit-testing tools like Spock and Spec2 (see section 10.4.3) provide built-in support for data tables. For example, using Spock (which we looked at briefly in chapter 2), the unit test in listing 10.1 might look like the following listing.

Listing 10.2 A data-driven unit test in Spock

```
class WhenEarningStatus extends Specification {

    def "should earn status based on the number of points earned"() {
        given:
        def member = FrequentFlyer.withFrequentFlyerNumber("12345678")
                .named("Joe", "Jones")
                .withStatusPoints(initialPoints)
                .withStatus(initialStatus);

        when:
        member.earns(earnedPoints).statusPoints()

        then:
        member.status == finalStatus

        where:
        initialStatus | initialPoints | earnedPoints | finalStatus
        Bronze        | 0             | 100          | Bronze
        Bronze        | 0             | 300          | Silver
        Bronze        | 100           | 200          | Silver
        Silver        | 0             | 700          | Gold
        Gold          | 0             | 1500         | Platinum
    }
}
```

Create a new Frequent Flyer member.

The member earns some points.

Check the status.

The test data used in the previous steps comes from here.

In both these examples, using data tables makes the unit test easier to read, extend, and maintain.

10.3.3 Discover new classes and services as you implement the production code

So far, we've looked at several ways to describe what you'd like your application code to look like and how it should behave. You've described how you want to be able to upgrade a member's status in the code, but you haven't written any corresponding